



LEER



SESIÓN 07 · UNIDAD II · ETHEREUM Y CONTRATOS

LEER UN CONTRATO QUE NO ESCRIBISTE

Si no puedes responder las cuatro pasadas sobre tu propio contrato, ese contrato todavía no es tuyo.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Cinco minutos de apertura, tres bloques y una pausa de 10 minutos entre B y C. Hoy no se escribe un contrato nuevo: se lee uno ajeno, se intenta romper —y se rompe— y después cada equipo lee el suyo.

A

LAS CUATRO PASADAS

Leer en vivo Entradas.sol, el contrato del proyecto del docente: qué promete, quién manda, por dónde entra y sale el dinero, qué pasa si...

~55 min

B

LABORATORIO DE ATAQUE

Un encargo de una línea: revendan una entrada por encima del tope. Seis ataques, una vulnerabilidad real y los cinco tipos que hay detrás.

~70 min

C

SU PROPIO CONTRATO

Las cuatro pasadas aplicadas al contrato del proyecto, aquí, con la ficha de lectura en la mano.

~40 min



AL SALIR DE AQUÍ

Leer un contrato ajeno con método, encontrar lo que su autor no dijo, y defender el contrato del proyecto propio.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Qué promete un contrato, leyendo solo los nombres de sus funciones y eventos?
- 02** ¿Quién manda adentro, y qué es exactamente lo que NO puede hacer?
- 03** ¿Por dónde entra el dinero, dónde se queda y por dónde sale?
- 04** ¿Qué regla hace cumplir de verdad, y dónde termina su poder?

RA3 · patrones y estándares (se evalúa) · RA5 · base de la seguridad (complementario, se profundiza en la S9)



SIGLAS Y TÉRMINOS DE HOY

LAS CUATRO PASADAS*el método de hoy*

Qué promete · quién manda · por dónde el dinero · qué pasa si. En ese orden, sin saltarse ninguna.

ERC-721*estándar de token no fungible*

Cada entrada es un token con dueño. OpenZeppelin trae la implementación: aquí se hereda, no se copia.

COMPUERTA*bandera que habilita un paso*

Una variable que una sola función abre, por un instante, y cierra. Hoy aparece una, y hubo que corregirla.

CEI*Checks-Effects-Interactions*

Validar, cambiar el estado propio y, solo al final, llamar hacia fuera.

GANCHO · HOOK*función interna que se reemplaza*

Un punto del padre por el que pasa siempre cierta operación. Sobreescribirlo cambia la regla para todas.

AVISO AL RECEPTOR*onERC721Received*

Al recibir un token, si el receptor es un contrato se ejecuta SU código. Hoy eso tiene consecuencias.

ERROR PERSONALIZADO*revert con datos adentro*

revert PrecioSobreTope(tope, pedido): dice qué pasó y con qué números. Más barato que una cadena.

PATRÓN DE RETIRO*acreditar en vez de enviar*

El contrato anota el saldo y cada quien llama retirar(). Nadie depende de que un envío ajeno funcione.



A



APROXIMADAMENTE 55 MINUTOS · EN VIVO

LEER UN CONTRATO QUE NO ESCRIBISTE

Cuatro pasadas, en orden, sobre un contrato real de 254 líneas. En pantalla, en vivo, y sin que nadie lo haya escrito en esta clase.

CUATRO PASADAS, EN ESTE ORDEN

Nadie lee un contrato de corrido y entiende. Se lee cuatro veces, y cada vez se busca una sola cosa. Saltarse una pasada es la forma más común de creer que se entendió.

- | | | |
|-----------|------------------|---|
| 01 | ¿QUÉ PROMETE? | Solo los nombres: funciones públicas y eventos. Sin entrar a ningún cuerpo. |
| 02 | ¿QUIÉN MANDA? | Las direcciones privilegiadas. Qué puede cada una y, sobre todo, qué NO puede. |
| 03 | ¿Y EL DINERO? | Por dónde entra, dónde se queda mientras está adentro y por dónde sale. |
| 04 | ¿QUÉ PASA SI...? | La pasada del adversario. Es la única que no se responde leyendo, y abre el bloque B. |

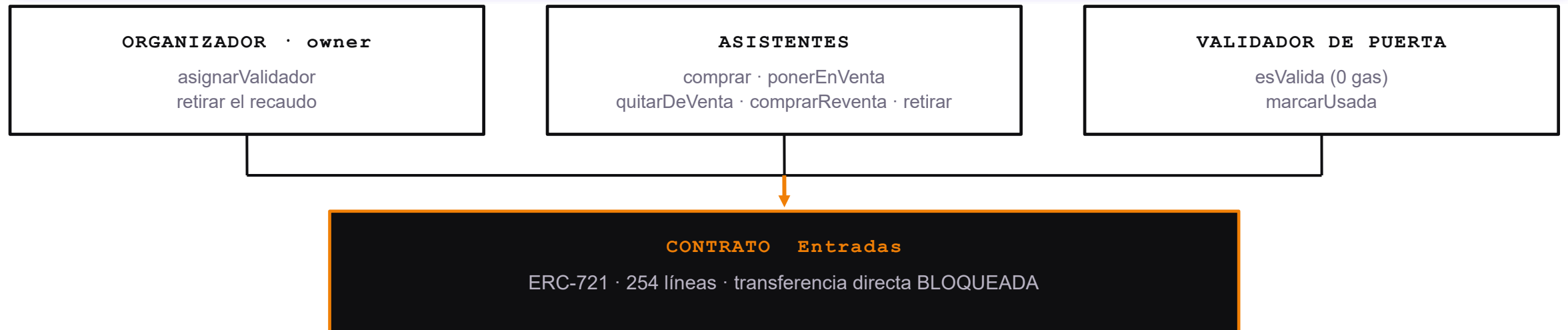
🗨 LA VARA DE HOY, Y LA DE LA SESIÓN 8

Si no pueden responder las cuatro pasadas sobre su propio contrato, ese contrato todavía no es suyo. Da igual quién escribió las líneas.



EL CONTRATO QUE VAMOS A LEER HOY

La regla de reventa deja de ser un término y condiciones: pasa a ser algo que no se puede incumplir.



Es la entrega 2 del proyecto del docente, publicada en github.com/andresdapena81/entradas-reventa-controlada. Hoy es el paciente: se lee en pantalla y no se toca.



PASADA 1: SOLO LOS NOMBRES

Regla de esta pasada: no se abre ningún cuerpo de función. Solo la lista de nombres públicos. Lo que se deduce de un nombre es una hipótesis, y se anota como hipótesis.

FUNCIÓN PÚBLICA	QUÉ PROMETE EL NOMBRE
<code>comprar() payable</code>	Alguien paga y recibe. Al ser payable y no llevar parámetro de precio, el precio lo fija el contrato, no el vendedor.
<code>ponerEnVenta(id, precio)</code>	El dueño ofrece. Que el precio sea un parámetro y no un valor fijo sugiere que hay un límite escrito en otra parte.
<code>quitarDeVenta(id)</code>	La oferta se puede deshacer. Una oferta irrevocable sería una trampa; esta no lo es.
<code>comprarReventa(id) payable</code>	Hay un mercado secundario ADENTRO del contrato. Y si está adentro, lo más probable es que afuera esté cerrado.
<code>retirar()</code>	El dinero no se envía: se reclama. Patrón de retiro reconocido sin abrir una sola línea del cuerpo.
<code>esValida(id, portador) view</code>	Una consulta, gratis, pensada para la puerta del evento: pregunta por un portador concreto.
<code>marcarUsada(id)</code>	Alguien escribe sobre la entrada de otro. ¿Quién tiene ese permiso? Esa pregunta es la pasada 2.



SIETE EVENTOS, Y LO QUE DELATAN

Los eventos son la única memoria del contrato que se lee cómodamente desde fuera. Quien quiera auditar, o construir una interfaz, vive de esta lista.

EVENTO	QUÉ CUENTA, Y A QUIÉN LE SIRVE
EntradaEmitida	Se vendió una del lote original: id, comprador y precio. Con esto se reconstruye el aforo sin leer el estado.
PuestaEnVenta · QuitadaDeVenta	El mercado secundario se puede seguir en vivo desde una interfaz, sin consultar el contrato.
Revendida	De quién a quién y por cuánto. Es el renglón que permite auditar si el tope se respetó.
Retirado	Quién sacó dinero y cuánto. La contabilidad del contrato queda publicada.
EntradaUsada	Quién entró al evento con cuál entrada, y cuándo.
ValidadorCambiado	Lleva el anterior Y el nuevo: se puede reconstruir la cadena de custodia del aparato de la puerta.

Lo que falta también habla: no hay ningún evento de pausa, de actualización ni de reembolso. Si algo importante no emite nada, casi siempre es porque no existe. Anótenlo y lo comprobamos al cerrar el bloque.

LO QUE HEREDA: ERC-721 Y OWNABLE

```
import {ERC721} from "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract Entradas is ERC721, Ownable {
    constructor(...) ERC721("Entrada USB", "ENT") Ownable(msg.sender) { ... }
}
```

LO QUE LLEGA HEREDADO	Y LO QUE ESO IMPLICA AL LEER
ERC721	ownerOf, balanceOf, approve, transferFrom, safeTransferFrom y varios ganchos internos. Son cientos de líneas que NO están en este archivo.
Ownable	owner(), onlyOwner y transferOwnership. Su error de permiso se llama OwnableUnauthorizedAccount y no figura entre los once de este contrato.

LA TRAMPA DE LEER UN CONTRATO QUE HEREDA

Buena parte de lo que hace Entradas.sol no está escrito en Entradas.sol. Cuando un nombre no aparece en el archivo, está en el padre, y hay que abrir el padre. Y al contrario: heredar código auditado y usado por miles de contratos es la defensa más barata que existe.



PASADA 2: DOS DIRECCIONES CON PODER

	OWNER · EL ORGANIZADOR	VALIDADOR · LA PUERTA
Quién la fija	El que despliega: Ownable(msg.sender) en el constructor.	La asigna el owner con asignarValidador, y la puede cambiar cuando quiera.
Qué PUEDE	Asignar el validador · marcar una entrada como usada · retirar su propio saldo del recaudo.	Marcar una entrada como usada. Nada más.
Qué NO puede	Emitir gratis · mover una entrada ajena · cambiar el tope · tocar el saldo de otro · parar el contrato.	Emitir · mover entradas · ver ni tocar dinero · cambiar nada del contrato.
Si se pierde la clave	Nadie asigna validador y el recaudo acreditado queda atrapado. Las entradas siguen funcionando.	El owner asigna otro aparato en una transacción. Ese es justamente el punto.

LA DECISIÓN: UN VALIDADOR DÉBIL A PROPÓSITO

El aparato de la puerta se presta y se pierde. Por eso el validador no emite, no mueve entradas y no toca dinero: quien se lo encuentre no puede hacer nada con valor. El límite honesto: puede quemar la entrada de alguien que todavía no llegó.

LA ALTERNATIVA DESCARTADA

Una sola aplicación con permisos por rol. Más cómoda de construir, y convierte el aparato más expuesto del evento en el que firma transacciones de valor. Está en ARQUITECTURA.md, sección 6, con las otras cinco decisiones.



REGLAS QUE NADIE PUEDE CAMBIAR

```
uint256 public immutable aforo;           // se fija al desplegar
uint256 public immutable precioOriginal;   // y ya nadie lo cambia
uint256 public immutable topeReventa;     // <- la regla del laboratorio de hoy
uint256 public immutable cierreReventa;

uint256 public emitidas;                  // normal: cambia con cada compra
address public validador;                 // normal: el owner la reasigna
```

PALABRA	DÓNDE VIVE EL VALOR	EN ENTRADAS.SOL	ERROR TÍPICO
constant	Incrustado en el código al compilar. No ocupa ranura.	No hay ninguna: todo depende del despliegue.	Querer asignarla en el constructor: no compila.
immutable	Se fija en el constructor y queda incrustado en el código desplegado.	aforo, precioOriginal, topeReventa, cierreReventa.	Usarla para algo que deba cambiar después.
normal	Una ranura de almacenamiento. Leerla cuesta 2 100 de gas la primera vez en cada transacción.	emitidas, validador, evento y los tres mappings.	Dejar normal algo que nunca cambia: se paga en cada lectura.

Lo que se lee aquí es una promesa fuerte: el tope de reventa NO se puede cambiar, y el organizador tampoco puede. Esa es la diferencia entre una regla y una intención. (El costo de 2 100 es el de la EIP-2929.)



PASADA 3: LAS DOS PUERTAS DEL DINERO

```
function comprar() external payable returns (uint256 tokenId) {  
    if (emitidas >= aforo) revert AforoAgotado();  
    if (msg.value != precioOriginal) revert PagoIncorrecto(precioOriginal, msg.value);  
  
    tokenId = ++emitidas;  
    saldos[owner()] += msg.value;        // se ACREDITA, no se envía  
    _safeMint(msg.sender, tokenId);  
    emit EntradaEmitida(tokenId, msg.sender, msg.value);  
}
```

01

PUERTA 1 · COMPRAR()

payable y msg.value exacto. Entra precioOriginal y se acredita al organizador. No hay descuentos ni cortesías: esa función no existe.

02

PUERTA 2 · COMPRARREVENTA()

payable y msg.value igual al precio de la oferta. Entra y se acredita al vendedor, nunca al contrato ni al organizador.

Y no hay una tercera. El contrato no tiene receive ni fallback, así que el ether que llegue sin llamar a una de esas dos funciones se rechaza. Eso también es una decisión, y se lee por ausencia.



DÓNDE SE QUEDA: STRUCT Y MAPPING

```
struct Oferta { uint256 precio; bool activa; }

mapping(uint256 tokenId => Oferta) public ofertas;    // qué se ofrece y a cuánto
mapping(uint256 tokenId => bool)    public usada;      // si ya entró al evento
mapping(address titular => uint256 monto) public saldos; // lo que cada quien reclama
```

PROPIEDAD DEL MAPPING	CÓMO SE LEE EN ESTE CONTRATO
Toda clave existe	ofertas[999] devuelve Oferta{0, false} aunque la entrada 999 no exista. Por eso hay un campo activa: distingue «sin oferta» de «ofrecida en cero».
No se puede recorrer	No hay forma de listar las entradas en venta desde el contrato. La interfaz las arma leyendo los eventos PuestaEnVenta.
Borrar es poner en cero	delete ofertas[id] deja activa en false. Es lo que hace comprarReventa en cuanto cobra, y marcarUsada al entrar.
saldos es la contabilidad	Un solo mapping lleva el recaudo del organizador y lo de cada revendedor. El contrato no distingue de quién es: solo debe.

Error típico al leer: ver un mapping y suponer que hay una lista. No hay. Si una interfaz les muestra «todas las entradas en venta», ese dato no salió del estado del contrato: salió de los eventos.



POR DÓNDE SALE: RETIRAR() Y CEI

```
function retirar() external {
    uint256 monto = saldos[msg.sender];
    if (monto == 0) revert SinSaldo();           // 1 · CHECKS

    saldos[msg.sender] = 0;                      // 2 · EFFECTS

    (bool ok, ) = payable(msg.sender).call{value: monto}(""); // 3 · INTERACTIONS
    require(ok, "el envio fallo");
    emit Retirado(msg.sender, monto);
}
```

POR QUÉ ACREDITA Y NO ENVÍA

Si comprarReventa le enviara el dinero al vendedor, la venta entera dependería de que ese vendedor quisiera recibirlo. Un contrato vendedor que reviente al recibir congelaría el mercado para todos.

POR QUÉ EL CERO VA ANTES DEL CALL

Cuando el receptor recibe el control, su saldo ya es cero. Si vuelve a entrar, no hay nada que llevarse. Es la reentrada, y se desactiva sola con el orden de las tres líneas.

Por qué call y no transfer: transfer entrega 2 300 de gas fijos, y en 2019 la EIP-1884 subió el costo de leer almacenamiento y dejó esa cifra corta. Una defensa que depende de un número que no controlas no es una defensa.

ERRORES QUE DICEN QUÉ PASÓ

```
error PagoIncorrecto(uint256 esperado, uint256 recibido);
error PrecioSobreTope(uint256 tope, uint256 pedido);
error TransferenciaDirectaBloqueada();

if (precio > topeReventa) revert PrecioSobreTope(topeReventa, precio);
```

LOS ONCE ERRORES, POR GRUPO	QUÉ LE DICEN A QUIEN LEE
Pago	PagoIncorrecto(esperado, recibido) y PrecioSobreTope(tope, pedido). Llevan los dos números: quien falla ve en qué se equivocó sin consultar nada.
Permiso	NoEsPropietario · NoAutorizado · NoSePuedeComprarASiMismo. Marcan los tres sitios donde el contrato comprueba quién está llamando.
Momento	ReventaCerrada · AforoAgotado · EntradaYaUsada · NoEstaEnVenta. El ciclo de vida completo de una entrada se deduce de esta lista.
La regla	TransferenciaDirectaBloqueada. El único error que no corresponde a ninguna función pública: salta desde un sitio que todavía no hemos leído.

Esa última fila es la pista que abre el laboratorio: hay una regla que ninguna función pública declara y que aun así existe. Guárdenla.



LO QUE CUESTA, MEDIDO EN GAS

Camino completo del dinero, con recibos reales: desplegar, dos compras, poner en venta, una reventa y los retiros. Red local, solc 0.8.34, optimizador en 200.

LO QUE HACE UNA TRANSACCIÓN	GAS MEDIDO	DE DÓNDE SALE EL NÚMERO
<code>desplegar el contrato</code>	1 889 937	7 999 bytes de código. Se paga una sola vez, y la paga el organizador.
<code>comprar()</code> · la primera entrada	117 309	Acuña el token y estrena la ranura del saldo del organizador.
<code>comprar()</code> · la segunda	83 109	34 200 menos que la primera, sin hacer nada distinto.
<code>ponerEnVenta()</code> en el tope	72 090	Escribe la oferta: el precio y la bandera, dos ranuras nuevas.
<code>comprarReventa()</code>	81 839	Cobra, acredita al vendedor, borra la oferta y mueve el token.
<code>retirar()</code>	30 023	Igual para el organizador y para el revendedor: es la función más barata.

Lo que enseña la tercera fila: esos 34 200 de diferencia no son el token. Son las escrituras que la primera compra estrena y la segunda ya encuentra hechas. Estrenar almacenamiento es lo más caro que hace un contrato.



PASADA 4: LA PASADA DEL ADVERSARIO

La cuarta pasada no busca errores de código: busca supuestos. Por cada regla que el contrato promete, se pregunta quién gana rodeándola y qué tendría que hacer para lograrlo.

- | | | |
|-----------|--------------------------|---|
| 01 | ¿QUIÉN GANA SI SE ROMPE? | El revendedor que puede cobrar más del tope. Tiene motivo, tiene tiempo y, si sabe programar, tiene herramientas. |
| 02 | ¿QUÉ SUPONE EL CONTRATO? | Que la única forma de mover una entrada es comprarReventa. Si ese supuesto es falso, el tope es decorativo. |
| 03 | ¿HASTA DÓNDE LLEGA? | Solo a lo que pasa por sus funciones. Lo que ocurra fuera de la cadena el contrato no lo ve, y no puede verlo. |
| 04 | ¿Y SI NO USAN LA APP? | Nadie está obligado a usar la interfaz oficial. Un atacante llama al contrato directamente, o escribe el suyo. |

Estas preguntas no son un ejercicio: aplicadas a este contrato, esta semana, encontraron un hueco de verdad. Lo van a encontrar ustedes en el bloque B, corriendo código.



LO QUE EL CONTRATO NO TIENE

LO QUE NO ESTÁ, Y SE NOTA AL LEER	QUÉ SIGNIFICA QUE NO ESTÉ
No es pausable	No hay Pausable ni interruptor de emergencia. Si mañana aparece un fallo, nadie puede detener las ventas: solo esperar el cierre de la reventa.
No es actualizable	No hay proxy. El código desplegado es el que queda para siempre. El tope es inamovible, y los errores también.
No tiene receive ni fallback	El ether que llegue sin llamar a comprar o a comprarReventa se rechaza. Una sola puerta por donde entra el dinero.
No tiene reembolsos	No existe devolver una entrada al organizador. Si el evento se cancela, el contrato no sabe resolverlo.
No tiene lista de ofertas	Ninguna función devuelve «las entradas en venta». Eso lo arma la interfaz leyendo eventos.



LO QUE ESTA LISTA VALE EN UNA DEFENSA

Un ausente no es un olvido hasta que se demuestre. En la Sesión 8 la pregunta va a ser esta: ¿esto no está porque lo decidieron, o porque nadie se dio cuenta? Las dos respuestas se oyen distinto, y solo una se puede defender.



LO QUE LAS CUATRO PASADAS DEBIERON DEJAR

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 Sin abrir ningún cuerpo de función: ¿qué hace sospechar que este contrato tiene un tope de precio?

02 El organizador pierde su clave privada. ¿Qué deja de funcionar, y qué sigue funcionando igual?

03 En retirar(), ¿qué línea va antes del call y qué ataque evita exactamente?

04 Nombren dos cosas que el contrato NO tiene, y qué consecuencia tiene cada ausencia.

B



APROXIMADAMENTE 70 MINUTOS · EN PAREJAS

EL LABORATORIO DE ATAQUE

Un encargo de una sola línea: revendan una entrada por encima del tope. Seis ataques, una vulnerabilidad real encontrada esta semana, y los cinco tipos que hay detrás.



EL ENCARGO, EN UNA SOLA LÍNEA

Revendan una entrada por encima del tope.

01 NO SE TOCA EL CONTRATO

Entradas.sol y EntradasVulnerable.sol son copias de lectura. Atacar cambiando el contrato que se ataca no demuestra nada.

02 SE ATACA DESDE AFUERA

Como lo haría alguien de verdad: con otro contrato, llamando a las funciones públicas sin pasar por la interfaz.

03 LA PRUEBA ES LA EVIDENCIA

Un intento sin una prueba que lo corra es una opinión. Si el intento funciona, tiene que funcionar en verde.

Archivos del laboratorio: contracts/s07/Entradas.sol · EntradasVulnerable.sol · AtacanteTope.sol · AtacanteCompuerta.sol, y las pruebas test/s07/AtaqueAlTope.test.js y CompuertaAbierta.test.js.



QUINCE PRUEBAS, Y TODAS EN VERDE

```
npx hardhat test test/s07/AtaqueAlTope.test.js test/s07/CompuertaAbierta.test.js
```

```
S07 · el revendedor programa su propio contrato
  ✓ ★ intento 1 · transferFrom directo: BLOQUEADO
  ✓ ★ intento 2 · safeTransferFrom: BLOQUEADO igual
  ✓ ★ intento 3 · aprobar a un cómplice: la aprobación SE CONCEDE...
  ✓ ★ intento 4 · sobre el tope: RECHAZADO · y el borde EN el tope: aceptado
  ✓ ★ intento 5 · vender en el tope y cobrar la diferencia POR FUERA
S07 · el ataque de la compuerta · versión VULNERABLE
  ✓ ★ el atacante saca su entrada del tope desde onERC721Received
S07 · el mismo ataque · versión CORREGIDA
  ✓ ★ la compuerta se cierra antes del aviso, y el escape falla
15 passing
```



QUINCE EN VERDE, Y ESO ES LO RARO

Una suite verde normalmente significa «funciona». Aquí significa otra cosa: unos intentos fallan como se espera, otros funcionan como NO se espera, y uno de ellos encontró un hueco de verdad en el contrato del docente.

Primero se corre, después se lee. Si una pareja no ve «15 passing», el problema es de entorno y se resuelve ahora: `npm install` desde `laboratorios-evm`.

UN REVENDEDOR QUE SABE PROGRAMAR

```
contract AtacanteTope is IERC721Receiver {
    function intentoTransferenciaDirecta(uint256 id, address comprador) external {
        IERC721(address(entradas)).transferFrom(address(this), comprador, id);
    }
    function intentoTransferenciaSegura(uint256 id, address c) external { ... }
    function intentoAprobarAComplice(uint256 id, address complice) external { ... }
    function intentoPrecioSobreTope(uint256 id, uint256 precioAbusivo) external { ... }
    function ponerEnVentaEnElTope(uint256 id) external { ... }
}
```

Cinco intentos, uno por función. No usa la interfaz oficial: llama al contrato directamente, que es lo que hace cualquiera con malas intenciones y una tarde libre.

INTENTO	LA IDEA DEL REVENDEDOR
1 · <code>transferFrom</code>	«Le paso la entrada a mi comprador y que él me pague por fuera.»
2 · <code>safeTransferFrom</code>	«Probemos con la versión segura, por si la otra está vigilada.»
3 · <code>approve</code>	«Apruebo a un cómplice y que la saque él.»
4 · <code>ponerEnVenta</code>	«Pongo el precio que yo quiera: 0,5 ETH en vez de 0,12.»
5 · <code>ponerEnVentaEnElTope</code>	«La pongo en el tope exacto, que es perfectamente legal.» Léanlo entero antes de opinar.



INTENTOS 1 Y 2: CONTRA UN MURO

```
it("★ intento 1 · transferFrom directo: BLOQUEADO", async () => {  
  await expect(atacante.intentoTransferenciaDirecta(tokenId, victima.address))  
    .to.be.revertedWithCustomError(entradas, "TransferenciaDirectaBloqueada");  
});
```

Las dos formas estándar de mover un ERC-721 revierten. Y ninguna función pública de Entradas.sol declara ese error.

LO QUE EL ATACANTE ESPERABA

transferFrom es parte del estándar ERC-721. Cualquier contrato que lo cumpla la tiene, y funciona. Entradas.sol la tiene y no funciona.

LO QUE ESO IMPLICA

Alguien sobrescribió algo heredado. El bloqueo no está en las funciones que leímos en la pasada 1: está más abajo, en una pieza del padre.



PREGUNTA

Ninguna de las funciones públicas que leímos declara TransferenciaDirectaBloqueada. ¿Desde dónde revierte, entonces?

Pista: la respuesta está en lo que el contrato hereda, no en lo que escribe.

EL GANCHO POR DONDE PASAN TODAS

LA VERSIÓN PUBLICADA HASTA ESTA SEMANA

```
function _update(address to, uint256 tokenId, address auth)
    internal override returns (address)
{
    address from = _ownerOf(tokenId);
    if (from != address(0) && !_transferenciaAbierta) {
        revert TransferenciaDirectaBloqueada();
    }
    return super._update(to, tokenId, auth);
}
```

DENTRO DE COMPRARREVENTA

```
_transferenciaAbierta = true;
_safeTransfer(vendedor, msg.sender, tokenId, ""); // la única vía
_transferenciaAbierta = false;
```

- Todo movimiento de un ERC-721 pasa por `_update`: acuñar, transferir y quemar.
- Se permite acuñar, que es cuando `from` vale cero.
- Se permite mover solo si la compuerta está abierta.
- La abre una sola función, por un instante, y solo tras verificar el tope.

Esa es la diferencia entre una regla escrita en unos términos y condiciones y una regla que el sistema hace cumplir.



INTENTO 3: APROBAR NO ES MOVER

LA APROBACIÓN SÍ SE CONCEDE

`approve(complice, id)` pasa sin problema, y `getApproved(id)` devuelve la dirección del cómplice. Tiene sentido: aprobar no mueve nada, solo anota un permiso en un mapping.

Y NO LE SIRVE DE NADA

Cuando el cómplice llama `transferFrom`, el gancho se interpone igual. Da exactamente lo mismo quién llame: la compuerta no está abierta.

```
await atacante.intentoAprobarAComplice(tokenId, complice.address);
expect(await entradas.getApproved(tokenId)).to.equal(complice.address); // pasa
await expect(entradas.connect(complice).transferFrom(atacante, victima, id))
    .to.be.revertedWithCustomError(entradas, "TransferenciaDirectaBloqueada");
```

[] LA LECCIÓN DE LECTURA

Un permiso concedido no es un movimiento permitido. Al leer un contrato hay que separar las dos cosas: `approve` escribe en un mapping, y mover es otra operación, que es la que está vigilada.



INTENTO 4: EL TOPE, Y SU BORDE

```
// el tope es 0,12 ETH y el revendedor pide 0,5
await expect(atacante.intentoPrecioSobreTope(tokenId, abusivo))
  .to.be.revertedWithCustomError(entradas, "PrecioSobreTope")
  .withArgs(TOPE, abusivo); // el error trae los dos números adentro
```

POR ENCIMA DEL TOPE: RECHAZADO

ponerEnVenta compara antes de escribir. El error no dice «no puedes»: dice cuál es el tope y cuánto se pidió. Quien lea el recibo entiende qué pasó.

JUSTO EN EL TOPE: ACEPTADO

Una prueba comprueba el borde exacto: precio igual a topeReventa entra. La comparación es «mayor que», no «mayor o igual». Esa diferencia de un carácter es una regla de negocio.



LO QUE HAY QUE MIRAR SIEMPRE EN UNA COMPARACIÓN

El borde. En un contrato, > y >= separan «se puede» de «no se puede». Cuando lean su propio contrato, busquen cada comparación y pregunten qué pasa exactamente en el valor límite.



LA COMPUERTA SE QUEDABA ABIERTA

Preparando esta sesión, las cuatro pasadas se aplicaron al contrato del propio docente. La pasada 4 encontró un hueco real: la promesa central del contrato —«el tope no se puede rodear»— era falsa. Estuvo publicada así hasta el 7 de octubre de 2026.

- | | | |
|-----------|--------------------------|--|
| 01 | ABRE LA COMPUERTA | <code>comprarReventa</code> pone <code>_transferenciaAbierta</code> en <code>true</code> para poder mover el token. Hasta aquí, todo correcto. |
| 02 | AVISA AL RECEPTOR | <code>_safeTransfer</code> llama a <code>onERC721Received</code> del comprador. Si el comprador es un contrato, SU CÓDIGO CORRE AHÍ. |
| 03 | Y SIGUE ABIERTA | Porque se cerraba después, al volver. Durante ese instante, cualquier transferencia pasa el control de <code>_update</code> . |
| 04 | EL ESCAPE | Desde su propio aviso, el atacante llama <code>transferFrom</code> y saca OTRA entrada suya. Sin oferta, sin precio y sin tope. |

EL ESCAPE, DENTRO DEL AVISO

```
function onERC721Received(address, address, uint256, bytes calldata)
    external returns (bytes4)
{
    // la compuerta del contrato de entradas sigue abierta en este instante
    try IERC721(address(entradas)).transferFrom(address(this), destino, tokenAEscapar) {
        loLogro = true; // y lo logró
    } catch { loLogro = false; }
    return IERC721Receiver.onERC721Received.selector;
}
```

LAS CUATRO PRUEBAS DEL ATAQUE	RESULTADO
Contra EntradasVulnerable	El escape FUNCIONA: la entrada 1 acaba en manos del comprador del atacante, y saldos del atacante queda en cero. El contrato no registró ningún precio.
Contra Entradas, la corregida	El escape FALLA: loLogro queda en false y la entrada 1 sigue siendo del atacante.
La compra que lo disparaba	Se completa igual: la entrada 2 cambia de dueño con normalidad. La corrección no rompe el camino legítimo.
Una reventa normal	Sigue funcionando entre dos personas. Es la prueba que evita «arreglar» rompiendo otra cosa.



LA CORRECCIÓN: TRES LÍNEAS

ANTES · PUBLICADO

```
if (from != address(0)
    && !_transferenciaAbierta) {
    revert TransferenciaDirectaBloqueada();
}
// se cerraba al volver, en comprarReventa
```

AHORA · CORREGIDO

```
if (from != address(0)) {
    if (!_transferenciaAbierta)
        revert TransferenciaDirectaBloqueada();
    _transferenciaAbierta = false;    // UN SOLO USO
}
```

+161

gas de más por reventa: 81 839 contra 81 678

+4

bytes de código: 7 999 contra 7 995

+852

gas de más al desplegar, una sola vez



POR QUÉ ESTA LÁMINA ESTÁ EN EL DECK

Porque el contrato que el docente publicó como ejemplo de «la regla no se puede rodear» sí se podía rodear. Estaba bien escrito, revisado y con sus pruebas en verde, y tenía un hueco que costaba 161 de gas tapar. Enseñarlo vale más que esconderlo.



INTENTO 5: EL PAGO POR FUERA

01

POR FUERA

La víctima le transfiere 0,38 ETH al revendedor, de billetera a billetera. Para el contrato de entradas, eso no existe.

02

EN EL TOPE

El revendedor pone la entrada en venta en 0,12 exacto. Perfectamente legal, y así lo confirma la prueba del borde.

03

LA COMPRA

La víctima llama comprarReventa con 0,12. Todas las validaciones pasan: tope, cierre, pago exacto, dueño.

04

EL RESULTADO

Pagó 0,5 ETH por una entrada con tope de 0,12. Cuatro veces el tope, y el contrato no vio nada raro.

El contrato hace cumplir la regla adentro. Con un pago por fuera no puede hacer nada, y nunca va a poder.



DÓNDE TERMINA EL PODER DEL CONTRATO

LO QUE ESTO NO ES

No es un fallo del contrato, y no se parece al hallazgo de la compuerta. Ahí faltaba una línea; aquí no hay línea que escribir: ningún código puede ver un pago en efectivo.

LO QUE SÍ ES

El límite de lo que un contrato puede hacer cumplir. Lo que entra en su alcance lo hace cumplir sin excepciones; lo que queda fuera no lo roza siquiera.

LO QUE EL TOPE SÍ LOGRA	Y LO QUE NO
Mata el mercado abierto de reventa	No mata el acuerdo privado entre dos personas.
Deja el precio en cadena auditable	No dice nada del precio real que se pagó.
Pone un techo al precio	No pone un precio justo: el techo se vuelve el precio.
Cierra todo al llegar el cierre	No limita cuántas veces se revende antes.

Y esta es la tesis del curso en su versión incómoda: la pregunta nunca es «¿se puede poner en la cadena?». Es «¿qué parte del problema queda dentro del alcance de lo que la cadena hace cumplir, y qué parte sigue dependiendo de la gente?».

LOS CINCO TIPOS DE ATAQUE DE HOY

Cada intento de hoy pertenece a una familia conocida. Reconocer la familia es lo que les permite trasladar la lección a su propio contrato, que no tiene entradas ni topes.

EL INTENTO DE HOY	LA FAMILIA, Y QUÉ ENSEÑA
Llamada directa al contrato	Se rodea la interfaz oficial. Una regla que solo vive en la aplicación no existe: cualquiera llama al contrato sin pasar por ella.
Puerta alterna a lo mismo	transferFrom y safeTransferFrom mueven lo mismo. Por eso se controla el punto por donde pasa el ESTADO, no cada puerta. Se llama mediación completa.
Abuso de la delegación	Aprobar sí se concede, porque aprobar no mueve nada. El control va en el momento de ejercer el permiso, no en el de concederlo.
Devolución del control	El contrato llama a código ajeno con el estado a medio camino. Es la familia de la reentrada; aquí la variante no toca el dinero: toca la compuerta.
Canal lateral fuera de cadena	El pago por fuera, que el contrato no puede ver. No es un fallo: es el límite de lo que un contrato hace cumplir.

Dos de estas familias tienen su caso real propio en el curso: las aprobaciones ilimitadas de ERC-20 y el robo a Badger DAO llegan en la S10; la reentrada completa, con The DAO, en la S9.



LA REGLA QUE HAY QUE LLEVARSE

Un control se pone en el punto por donde pasa el estado, no en cada puerta de entrada. Y todo lo que entregue el control a código ajeno debe dejar el estado ya cerrado antes de hacerlo.



Y UNA FAMILIA MÁS QUE ESTE CONTRATO NO RESUELVE

comprarReventa solo exige el pago exacto y que el comprador no sea el vendedor. Nada impide que un bot vea una oferta recién publicada y la compre antes que la persona a la que el vendedor se la quería dejar. Es front-running, de la familia del MEV: se ve en la S9 y en la S14. El contrato no lo resuelve, y decirlo vale más que esconderlo.

Las dos mitades de la regla se aplican a su contrato así: (1) busquen la operación que cambia el estado y pongan el control AHÍ, no en las tres funciones que la llaman; (2) hagan la lista de cada línea de su contrato que llame a una dirección que no controlan —un envío de ether, un token ajeno, un aviso al receptor— y comprueben que antes de esa línea el estado ya quedó en su sitio.



SU SEXTO INTENTO

El laboratorio no termina al correr las quince pruebas. Termina cuando cada pareja escribe un intento propio y demuestra si falla o si funciona.

01 ELIJAN UNA FAMILIA

De la tabla B.14. Un ejemplo que no está probado: ¿y si el comprador de la reventa fuera un contrato sin onERC721Received?

02 ESCRÍBANLO

Una función más en su copia de un atacante y una prueba más en su copia del archivo de pruebas. La copia: nunca el original.

03 CÓRRANLO

Si revierte: anoten con qué error y desde qué línea del contrato sale. Si pasa: anoten qué supuesto rompieron y qué ganaron.

04 DEFIÉNDANLO

Dos frases: a qué familia pertenece su intento y qué demostró el resultado. Un intento que falla enseña tanto como uno que funciona.

Evidencia del bloque: la captura de «15 passing», su sexto intento con su prueba, y las dos frases que lo defienden.



LO QUE EL ATAQUE DEBIÓ DEJAR CLARO

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 ¿Por qué el bloqueo de la transferencia no está en ninguna función pública, y dónde sí está?

02 La compuerta seguía abierta mientras corría el código del comprador. ¿Por qué eso bastaba para romper el tope?

03 El pago por fuera funciona y no se puede arreglar. ¿En qué se diferencia del hallazgo de la compuerta?

04 Enuncien la regla general: ¿dónde se pone un control, y qué hay que hacer antes de llamar a código ajeno?



C



APROXIMADAMENTE 40 MINUTOS · POR EQUIPO

SU PROPIO CONTRATO

Las cuatro pasadas, aplicadas aquí y ahora al contrato que su equipo expone en la Sesión 8.
Con la ficha de lectura en la mano.

LAS CUATRO PASADAS, SOBRE LO SUYO

Cinco o seis equipos, cuarenta minutos. Cada equipo abre su contrato, hace las cuatro pasadas en orden y llena la ficha. El docente circula.

PASADA	QUÉ LLENAN EN LA FICHA	MIN
1 · ¿Qué promete?	La lista de funciones públicas y eventos, y una frase por cada una: qué promete ese nombre.	8
2 · ¿Quién manda?	Cada dirección privilegiada, con dos columnas: qué puede y qué NO puede. Y qué pasa si se pierde esa clave.	8
3 · ¿Y el dinero?	Por dónde entra, dónde se queda y por dónde sale. Si no entra dinero, escriban por qué no y qué custodia en su lugar.	8
4 · ¿Qué pasa si...?	Tres supuestos del contrato, con su familia de la tabla B.14, y quién gana si cada uno es falso.	10

 LA FICHA DE LECTURA

material/proyecto/ficha-lectura-contrato.pdf. Se llena a mano, en el salón, y se entrega firmada por el equipo. Es el insumo de la defensa de la Sesión 8: lo que no esté en la ficha no se puede alegar en la exposición.



LAS TRES PREGUNTAS DE LA DEFENSA

En la Sesión 8, a cada equipo se le hacen estas tres. Siempre, a todos, y no hay versión corta. No son una sorpresa: están aquí, hoy, por escrito.

01 ¿QUIÉN MANDA?

Señalen la línea donde se decide quién tiene privilegios. ¿Qué puede hacer esa dirección que nadie más puede, y qué pasa el día que se pierda?

02 ¿POR DÓNDE SALE EL DINERO?

Señalen la única línea por la que el valor sale del contrato. Si hay dos, expliquen por qué hay dos. Si no sale nunca, digan qué queda atrapado.

03 ¿QUÉ PASA SI...?

Nombren el supuesto más frágil de su contrato y quién gana si es falso. Después muestren la línea que lo defiende, o admitan que no hay ninguna.

Ninguna se responde con el deck ni con una búsqueda: se responden señalando una línea de SU archivo en la pantalla. Si nadie del equipo puede señalarla, el equipo no ha leído su contrato.



ESCRIBIR CONTRATOS CON AYUDA DE IA

El problema no es usar la IA. El problema es no poder defender lo que produjo.

LO QUE ES LEGÍTIMO

Pedirle código, pedirle que explique una línea, pedirle alternativas y pedirle que critique su propio resultado. Usarla para leer más rápido un contrato ajeno. Nada de eso es trampa, y en la industria se hace todos los días.

LO QUE NO SE SOSTIENE

Entregar líneas que nadie del equipo puede explicar. En la defensa no se pregunta quién escribió el código: se pregunta por qué está ahí. «Lo puso la IA» no es una respuesta a esa pregunta.



LA REGLA, SIN MORALINA

Si nadie del equipo sabe por qué una línea está ahí, esa línea se borra o se entiende. Las dos salidas valen y las dos exigen trabajo. La tercera —dejarla y esperar que no pregunten— es la que cuesta la nota.



CADA EQUIPO, ANTES DE SALIR DEL SALÓN

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 Sin mirar la ficha: ¿quién manda en su contrato, y qué es lo que NO puede hacer?

02 ¿Por dónde sale el valor de su contrato? Nombren la función exacta.

03 Digan un supuesto de su contrato, a qué familia pertenece, y quién gana si resulta falso.

04 ¿Queda alguna línea en su contrato que nadie del equipo pueda explicar? ¿Cuál?



LA PREGUNTA DE LA SEMANA

El hueco de la compuerta se tapó con tres líneas. El pago por fuera no se puede tapar con ninguna. ¿Cuál de las dos cosas es su supuesto más frágil?

PARA LA PRÓXIMA SESIÓN

- Laboratorio de hoy: captura de «15 passing», su sexto intento con su prueba, y las dos frases que lo defienden, con la familia a la que pertenece.
- Ficha de lectura del contrato del equipo, completa y firmada. Se entrega antes de la S8: es el insumo de la exposición.
- El laboratorio de la subasta (laboratorios-evm/guias/s07-subasta-patrones.pdf, 16 páginas) pasa a TRABAJO AUTÓNOMO y sigue siendo evidencia evaluable: las 23 pruebas en verde y la tabla de gas push frente a pull.
- Traer el portátil con Node.js 22 y el repositorio instalado: la S8 es en la terminal, y arranca con las exposiciones.



LO QUE SE LLEVAN DE HOY

Si no puedes responder las cuatro pasadas sobre tu propio contrato, ese contrato todavía no es tuyo.

Hoy leyeron un contrato ajeno con método, le encontraron un hueco real que su autor no había visto, y vieron también dónde termina su poder. La próxima sesión ese método se vuelve público: cada equipo defiende el suyo delante del curso.

SESIÓN 8 · EXPOSICIÓN DE CONTRATOS, HARDHAT Y PRUEBAS AUTOMATIZADAS